API DEI GIOCHI BM - VOLUME 5

API dei giochi bm, volume 5: `strumenti`, progetti e risorse

Cartucce, progetti e file di risorse, le funzioni usate dagli strumenti di sviluppo e come condividere gli asset tra i giochi.

Indice

Il file cartuccia 4

Che cosa fa 4

Perché conta 4

Da sapere 4

Attenzione 5

Provalo, passo per passo 5

Dal PC 5

Le sezioni di una cartuccia 6

I tipi di sezione 6

Da sapere 7

L'API per gli strumenti 8

Giochi e progetti nell'API 8

Tastiera e file 8

Leggere e scrivere cartucce 8

Audio per gli editor 9

Modelli 3D 10

Eseguire, fare il debug e passare da uno strumento all'altro 11

L'assistente 11

File di risorse e condivisione degli asset 15

I tipi di file 15

Che cosa fa 15

Da sapere 15

Attenzione 15

Che cosa fa l'integrazione 16

Provalo, passo per passo: sposta un modello da un gioco a un altro 16

Gli altri comandi 17

La struttura del progetto 18

Che cosa fa 18

Da sapere 18

Attenzione 18
Provalo, passo per passo: sposta un gioco nella struttura comune 19

Le schermate per gli sviluppatori 20
Che cosa fa 20
Da sapere 20
Attenzione 21
Provalo, passo per passo 21
Per un manuale 21

Punti aperti 22

LEGGIMI

Che cos'è. Il volume 5 dell'API dei giochi bm. Fa per te se costruisci strumenti per la console, o se impacchetti e condividi giochi.

Come si usa. I primi due capitoli dicono che cos'è un file cartuccia e che cosa contiene. Il terzo elenca ogni funzione usata dagli strumenti. Gli ultimi tre sono guidati: condividere asset, la struttura del progetto, le schermate.

Che cosa manca. Ogni lacuna è segnata con un'etichetta evidenziata DA COMPLETARE; nulla è inventato.

Il file cartuccia

Un gioco su bm è un solo file. Lo stesso contenitore ha tre forme, e conta quali strumenti possono modificarlo.

Estensione	Che cos'è	Gli strumenti di bm possono modificarlo?
`.bm`	un **gioco**: sta nella scheda Giochi del menu	No. Gli strumenti lo leggono e ne prendono codice e asset
`.b16`	anche un **gioco**: l'edizione a risorse limitate per i portatili	No, come per il `.bm`
`.bme`	un **progetto**: lo stesso contenitore, quello che gli strumenti modificano	Sì

Che cosa fa

Un progetto è un `.bme` con il bit 0 dell'u16 all'offset 18 dell'intestazione acceso. Un `.b16` è segnato con il bit 1 dello stesso u16. Nient'altro nel contenitore cambia. Un progetto produce il suo gioco con una compilazione; un gioco produce una copia modificabile di sé stesso.

Perché conta

Uno strumento non sovrascrive mai il tuo gioco per sbaglio. Modifichi il progetto e compili il gioco quando è pronto.

Da sapere

- Uno strumento che salva un gioco chiede prima: *Save an editable copy: NAME.BME?* La copia va in `/bme` (`/bme/NAME.BME` , oppure `NAME1.BME` se il nome è occupato). Un `.bme` accanto al suo gioco su una scheda vecchia si legge ancora e si salva dove sta.
- Nel menu, *Make an editable copy* è nelle opzioni di un gioco. Il progetto non è nelle schede del menu: aprilo dall'SDK con *Open* (**Ctrl+O**).
- *Build the game (.bm)* è nelle opzioni di un progetto. Nell'SDK il tasto è **Ctrl+B**.
- Una compilazione di un progetto in `/bme` scrive il gioco nella cartella dei giochi (`/carts` sul Pi, `/bm` sull'RGB30); un progetto accanto al suo gioco compila accanto. Il gioco ha il nome del progetto salvato all'offset 104 dell'intestazione. La compilazione successiva sostituisce quel gioco e solo quello. Un gioco con lo stesso nome che non è stato compilato dal progetto resta, e il nuovo prende una cifra (`PONG1.BM`).

- Il `.bm` e il `.b16` sono edizioni separate. Un `.b16` non sostituisce mai il `.bm`, e i sorgenti del `.bm` non vengono mai sovrascritti per fare il `.b16`.
- Un file `.b16` non ha un limite di dimensione proprio: gli strumenti di build non rifiutano più un file grande. Il Market accetta fino a 100 MiB a file. Il suo schermo è 360x360 (o 720x720, lo dice il profilo), uguale su Pi e su RGB30.
- Le altre cartucce scrivono in `/carts` solo file `.bm` e `.bme`, e solo quelli che hanno creato nella stessa esecuzione.

Attenzione

- [DA COMPLETARE: stato del profilo `.b16`. `docs/B16.md` dice che il profilo è deciso ma non ancora implementato, e il §3 descrive un contenitore `B16CART` diverso che il §0 sostituisce con il contenitore `.bm`; `src/bm/project.h` e `scripts/mkbm.py` segnano ed esportano già i file `.b16` (bit 1 all'offset 18, controllo degli 8 MiB). Dire quale delle due è quella attuale.]
- Le voci *Build .bm* ed *Export .b16* dell'SDK chiamano `bm_build_b16`; **Ctrl+Shift+B** nell'SDK di bm esporta sempre un `.b16`.

Provalo, passo per passo

1. Sulla console apri la scheda Giochi e seleziona un gioco.
2. Premi **X** sul pad dell'RGB30 (**C** sul Pi) per aprire le sue opzioni e scegli *Make an editable copy*.
3. Apri la scheda Dev e avvia **bm SDK**. La scheda non elenca i progetti.
4. Nell'SDK premi **Ctrl+O** e scegli il progetto.
5. Nell'SDK premi **Ctrl+B** (*Build the game .bm*). Il gioco compare nella scheda Giochi.

[DA COMPLETARE: shots/api-tools-editable-copy.png]

PROVA

Compila due volte lo stesso progetto. La seconda compilazione sostituisce il primo gioco: non ottieni un secondo file.

Dal PC

`mkbm.py` scrive il tipo di file in base all'estensione di `-o`: `.bm`, `.bme` o `.b16`.

DA COPIARE

```
python3 scripts/mkbm.py -o ball.bme --lua carts/ball/main.lua --title "Ball" --author "me"
python3 scripts/mkbm.py --from ball.bm --b16
```

ATTENZIONE

Nessun controllo rifiuta un `.b16` grande in fase di build. Con `--from` e `--b16` e senza `-o`, il file viene scritto accanto al sorgente con l'estensione `.b16`.

Le sezioni di una cartuccia

Una cartuccia è un'intestazione di 128 byte, una tabella delle sezioni e i corpi delle sezioni. Tutto è little endian.

Offset	Contenuto
0	magic `BMCART` e due byte a zero (`BM33CART` nei file vecchi, ancora letti)
8	u16 versione (1)
10	u16 dimensione dell'intestazione (128)
12	u16 larghezza, u16 altezza: 640x360, 480x270, 320x180, 256x256, 360x360
16	u8 formato pixel (1 = RGB565)
17	u8 numero di sezioni
18	u16 flag: bit 0 progetto, bit 1 `.b16`
20	u32 CRC-32 di tutto ciò che segue l'intestazione
24	char[48] titolo
72	char[32] autore
104	char[16] il progetto da cui è stato compilato un gioco; zero fino a 128
128	la tabella delle sezioni: per ogni sezione u32 tipo, u32 offset, u32 dimensione, u32 riservato

I tipi di sezione

Tipo	Nome	Che cosa contiene
1	LUA	lo script principale, sorgente UTF-8
2	SHEET	il foglio di sprite: u16 w, u16 h, poi pixel RGBA8888 (alfa sotto 128 è trasparente)
3	MAP	u16 w, u16 h, poi celle u16 (indice dello sprite, 0 = vuota)
4	COVER	l'immagine della cartuccia nel menu (`mkbm.py` la fa 88x88); prima nel file
5	SHEET8	il foglio con al massimo 256 colori e sequenze ripetute, molto più piccolo; una cartuccia ha SHEET o SHEET8, non entrambi

Tipo	Nome	Che cosa contiene
6	AUDIO	il banco dei suoni (`BMAU`): suoni, effetti, pattern, brani; suonato da `sfx()` e `music()`
8	MESH	modelli 3D, da 1 a 256, con il margine delle texture
9	ANIM	scheletri e animazioni dei modelli MESH
10	INFO	testo `chiave: valore` di al massimo 16 KiB sul file e le sue parti
11	SPRITES	zone con nome del foglio (da 1 a 1024), con fotogrammi e velocità
12	LAYERS	gli altri livelli della mappa (da 2 a 8, la sezione MAP è il livello 1)
13	FLAGS	gli 8 flag di ogni cella 8x8 del foglio
14	BOXES	hitbox e hurtbox dei fotogrammi delle zone SPRITES (da 1 a 4096)

Da sapere

- I tipi 6 e 7 erano MESH e ANIM nei primi file di bm Studio. Una sezione di tipo 6 che non è un banco dei suoni viene ancora letta come MESH, una di tipo 7 come ANIM. Si scrivono come 8 e 9.
- La grafica è salvata indipendentemente dal formato dello schermo e convertita quando la cartuccia viene caricata.
- Una sezione che il kernel non conosce viene conservata quando uno strumento riscrive il file.
- Flag dei tile usati da bmlib: 0 solido, 1 piattaforma, 2 scala, 3 acqua, 4 fa male.

L'API per gli strumenti

Parte dell'API dei giochi. Queste funzioni sono pensate per gli strumenti di sviluppo (gli editor della suite). I giochi non ne hanno bisogno, e il progetto in `docs/B16.md` dice che il profilo `.b16` lascia fuori dalla sandbox le funzioni `cart_*` e `ls`; la console non lo applica ancora.

Giochi e progetti nell'API

`cart_save`, `cart_write` e `cart_put_audio` restituiscono `true` **e il file scritto** quando non è quello richiesto. Succede quando salvi un gioco: lo strumento scrive la copia modificabile e prosegue con quel file. Un nuovo `.bm` diventa un progetto (`"/carts/NEW.BM"` diventa `/carts/NEW.BME`). Un progetto produce il suo gioco con `cart_build`.

Tastiera e file

Funzione	Descrizione
`keyheld(name)`	`true` finché il tasto `name` è premuto: da `"f1"` a `"f12"`, `"tab"`, `"space"`, `"enter"`, `"esc"`
`keyp()`	il prossimo tasto digitato: un carattere (`"a"`, `"\n"` Invio, `"\b"` Backspace, `"\t"`), un nome (`"up"`, `"down"`, `"left"`, `"right"`, `"home"`, `"end"`, `"pgup"`, `"pgdn"`, `"del"`, `"esc"`, da `"f1"` a `"f10"`), `"^s"` per Ctrl+S, `"^S"` per Ctrl+Shift+S (anche `"^i"`, `"^m"`); `nil` se non c'è nulla. F11 e F12 sono del sistema e non arrivano mai. Dalla prima chiamata la tastiera scrive e non funziona più come gamepad per `btn()`
`keyhelp(list, [title])`	mostra i tasti della cartuccia sotto quelli del sistema mentre **F12** è premuto. `list` è `{ {"tasti", "che cosa fanno"}, "sottotitolo", ... }`, tasti come in `prompt()` (minuscolo la tastiera, maiuscolo il pad). Restituisce quante voci nominano un tasto che il sistema tiene per sé (F11, F12, Ctrl+Esc, Ctrl+Shift+Esc): sono mostrate in rosso e nel log. `keyhelp(nil)` lo toglie
`ls([folder])`	i file della scheda SD: `{ {name=, size=, dir=}, ... }`

Leggere e scrivere cartucce

Funzione	Descrizione
`cart_load(path)`	apre un `.bm`: il suo foglio di sprite (con flag, zone e box), la mappa (con i livelli) e i modelli 3D (con gli scheletri) prendono il posto di quelli della cartuccia chiamante. Restituisce `{title, author, res, lua, sheet_w, sheet_h, map_w, map_h, layers, [palette]}`

Funzione	Descrizione
`cart_sheet([w, h])`	la dimensione del foglio di sprite del progetto; con `w` e `h` (multipli di 8, da 8 a 4096) lo ridimensiona: i pixel che ci stanno restano, i nuovi sono trasparenti, i flag restano con il loro tile
`cart_new()`	un foglio di sprite e una mappa vuoti (256x256, un livello), senza flag, zone, box o modelli
`cart_save(path, {title, author, res, lua})`	scrive un `.bm` con il codice dato e il foglio, i flag, le zone, i box, la mappa, i livelli, la copertina, il banco dei suoni, i modelli e gli scheletri correnti. Un nome 8.3, per esempio `"/bme/GAME.BME"`. Restituisce `true` (e il file scritto quando non è `path`), oppure `false` e un messaggio
`cart_build(project)`	compila il gioco di un progetto (`.bme`) in un `.BM` nella cartella dei giochi (accanto ad esso per un progetto che non è in `/bme`). Restituisce `true` e il percorso del gioco, oppure `false` e un messaggio
`cart_read(path)`	il codice e l'intestazione di un `.bm`: `{title, author, res, lua, size}`, senza toccare foglio e mappa del chiamante. Per gli editor con più file aperti
`cart_write(path, {[lua, title, author, res, from, sections]})`	cambia solo il codice (e i campi dati) di un `.bm`; foglio, mappa, copertina, banco dei suoni e sezioni sconosciute restano. Senza `lua` il codice resta. `from`: le altre sezioni vengono da un altro file ("salva con nome"); `from = false`: una cartuccia nuova. `sections`: `{[8] = byte MESH, [9] = byte ANIM}` (`false` le toglie), controllati prima. `sheet = true`: il foglio di sprite del progetto prende il posto di quello del file, come SHEET8 se ha al massimo 256 colori, con i colori di `palette` (`{0xRRGGBB, ...}`) per primi. Restituisce come `cart_save`
`cart_data(kind, [bytes])`	le sezioni MESH (`kind` 8), ANIM (9), SPRITES (11) e BOXES (14) del progetto come stringhe nel formato di `src/bm/bm.h`. Senza `bytes` le restituisce (`nil` se non ci sono); con `bytes` le sostituisce (`nil` o `""` le toglie) e restituisce `true`, oppure `false` e il motivo
`cart_config(key, [value])`	il testo di una chiave di `bm/config.txt`, oppure `nil`. Solo chiavi che iniziano con la prima parola del titolo del gioco in minuscolo e `_` (Overbit: `overbit_`); qualsiasi altra cosa è un errore. Con un valore la chiave viene cambiata e salvata (`""` la cancella). Valori fino a 127 caratteri, senza `;` o `,`

Audio per gli editor

Funzione	Descrizione
`cart_audio([path])`	il banco dei suoni di un `.bm` come stringa (`false` se non ne ha) e il suo titolo; senza percorso, il banco della cartuccia in esecuzione
`cart_put_audio(path, bank, [title, lua])`	mette il banco (una stringa; `nil` lo toglie) in un `.bm` , il resto come prima; se il file non c'è lo crea con quel titolo e quel codice. Restituisce come `cart_save`
`audio_bank(bank)`	da ora suona questo banco; `nil` : nessuno
`audio_pattern(p, bpm, swing)`	un pattern in ciclo (le anteprime degli editor)
`audio_play(v, sound, note, [vol], [fx], [ms])`	un suono del banco su una voce (le anteprime degli editor)

Modelli 3D

Funzione	Descrizione
`cart_meshes(path)`	le mesh che il codice di un `.bm` costruisce con `mesh()` , `mesh_sphere()` e `mesh_cube()` : `{ {name=, kind=, verts={x,y,z,...}, faces={a,b,c,colore,...}, [uv={...}]}, ... }` (indici da 1, colore `-1` = texture), e `nil` o il primo errore del codice. Il codice gira a parte, in uno stato Lua tutto suo, con un limite di istruzioni; le altre funzioni di bm non fanno nulla. Per bm Mesh
`mesh_reduce(record, triangles, [bones, [max_err]])`	meno triangoli per un modello della sezione MESH (collasso degli spigoli con quadriche, nessuna IA). `bones` è l'osso di ogni vertice, `max_err` si ferma prima di un collasso più costoso (0: nessun limite). Restituisce il record, le ossa dei suoi vertici (`nil` senza `bones`) e il numero di triangoli; `nil` e un messaggio se il record è rotto. Sul PC: `tools/bmreduce.py`
`picture3d(action, ...)`	un'immagine diventa un modello 3D tramite un servizio da immagine a 3D. `picture3d("providers")` elenca i servizi; `picture3d("ready", service)` dà `true` , oppure `false` e il perché (manca la chiave); `picture3d("start", picture, {provider=, polycount=})` avvia il lavoro su un `.png / .jpg` della SD o su un URL https e dà l'id del lavoro (o `nil` e un messaggio); `picture3d("status", id, service)` dà `"running", progress` oppure `"done", url` ; `picture3d("take", url, {name=, faces=, height=})` scarica il `.glb` e lo converte in `{record=, flat=, texture=, nv=, nf=, textured=}` . Il primo servizio è Meshy, con la chiave `meshy_key=...` in `bm/config.txt` . Le chiamate bloccano finché la rete lavora

Funzione	Descrizione
`cutout3d(picture, {name=, lathe=, height=, depth=, segments=, faces=})`	un modello dal contorno dell'immagine, fatto sulla console senza rete né IA: una sagoma spessa `depth`, oppure con `lathe = true` il mezzo contorno fatto girare attorno all'asse verticale in `segments` passi. `height` in blocchi (2), `faces` triangoli al massimo (1200). Restituisce la stessa tabella di `picture3d("take")`, oppure `nil` e un messaggio

Eseguire, fare il debug e passare da uno strumento all'altro

Funzione	Descrizione
`cart_arg()`	la tabella con cui è stato aperto uno strumento, oppure `nil`: `path` (il file su cui lavorare), `error` (l'errore del gioco appena provato), `back` (true quando torna da un gioco), `from` (il nome dello strumento che lo ha chiamato), `run` (i numeri del dev kit di quel gioco); la impostano `cart_run` e `cart_tool`
`cart_run(path, [options])`	esce, fa girare quel file e poi riapre la cartuccia che l'ha chiesto, con `cart_arg() = {path=, error=, back=true, run=}`. `run` contiene i numeri della prova: `{frames, secs, fps, ms, ms_max, slow, lua_kb, lua_peak_kb, data_kb, instr_max, tokens, tris, gpu}`
`cart_run(path, {breaks = {lines}, stop = true})`	una sessione del debugger: il gioco si ferma alle righe `breaks` del suo `main.lua`, a `breakpoint()` e, con `stop`, alla sua prima riga. Da fermo: F10 o A la riga successiva, F8 o X dentro la chiamata, Shift+F8 o Y fuori dalla funzione, F5 o Start prosegue, Esc o Select ferma il gioco
`breakpoint([why])`	in un gioco provato da uno strumento, il debugger si ferma qui, con il motivo nel titolo; negli altri giochi non fa nulla
`cart_tool(name, [path])`	esce e apre un altro strumento sullo stesso file: `"studio"`, `"animator"`, `"mesh"`, `"pixel"`, `"code"`, `"sdk"`, `"sound"`. Lo strumento lo trova in `cart_arg()`, con `from` = il nome dello strumento che l'ha aperto

L'assistente

Una piccola IA che gira sulla console. Capisce una domanda (italiano o inglese, anche con errori di battitura) e risponde con voci della sua base di conoscenza, oppure disegna la base di uno sprite. Non è un chatbot: una minuscola rete INT8 sceglie tra le voci che conosce. Non fa nulla finché non la chiami. Provala da **Dev > Assistant**.

Funzione	Descrizione
`ai.ask(question, [{n=5, ctx=word, kinds="api,howto"}])`	le voci migliori, la prima la più probabile: `{ {id=, title=, kind=, score=}, ... }`, e come secondo valore i microsecondi impiegati. `ctx`: la parola sotto il cursore. `kinds`: `api`, `howto`, `error`, `tip`, `sprite`
`ai.entry(id)`	una voce: `{id, kind, title, name, text, code, gen, see = {id, ...}}`
`ai.list([kinds])`	tutte le voci `{id, title, kind}`
`ai.lang([language])`	la lingua delle risposte: `"it"` o `"en"`, e se segue le domande (`true`). `"it"` / `"en"` la fissano, `"auto"` segue ogni domanda. Parte da `assist_lang` in `bm/config.txt` (`it`, `en`, `auto`)
`ai.near(word)`	il nome dell'API più vicino a una parola sbagliata (`"sprr"` da `"spr"`, 1), oppure `nil`
`ai.sprite(request, [{gen=, size=16, seed=1, outline=true, palette={...}}])`	la base di uno sprite: `{w, h, gen, name, seed, px = {0xRRGGBB o -1, ...}}`, riga per riga. Le parole scelgono la ricetta, i colori e la dimensione; un altro `seed` è una variante; con `palette` ogni pixel diventa il colore più vicino della palette
`ai.recipes()`	le ricette degli sprite `{id, name}`; `ai.recipes("mesh")` quelle 3D `{id, name, rigged}`
`ai.script(text)`	un modello scritto nel linguaggio delle parti (`mat`, `box`, `bx`, `tube`, `cyl`, `ell`, `prism`, `wedge`, `tf`, `bone`, `use`, `side`, `mirror`, `clip`, `key`, `turn`, `shift`, uno per riga): la stessa tabella di `ai.mesh`, oppure `nil` e l'errore (`"line 3: ..."`)
`ai.music(request, [{gen=, seed=1, key=, minor=, bpm=, bars=, inst=, context=}])`	musica per un banco dei suoni: ritmi, basi, linee di basso, arpeggi, melodie ed effetti sonori, 87 ricette. Le parole scelgono la ricetta, la tonalità, il tempo, la lunghezza e lo strumento. `context = {notes = {...}, bars = {...}}`: le note già presenti (MIDI) e la battuta di ciascuna. Restituisce `{gen, name, kind, bpm, swing, key, minor, meter, bars, echo, room, seed, chords, instruments, patterns, sfx}`. Anche su RGB30, dove le parole scelgono la ricetta senza la rete. `ai.music_recipes()` le elenca
`ai.mesh(request, [{gen=, seed=1, scale=1, rig=true}])`	la base di un modello 3D per bm Studio e bm Animator: `{gen, name, seed, faces, bones, clips}`, le stesse tabelle di `bm3d.lua`. 53 ricette (forme, oggetti, persone, animali, macchine); `"no skeleton"` nelle parole lascia fuori lo scheletro
`ai.checksum(question)`	il CRC-32 delle uscite della rete per una domanda: per i test

Piccole reti per i giochi

Una rete di strati densi con pesi e attivazioni INT8, da una stringa scritta da uno script di addestramento (`scripts/nnetlib.py`).

Funzione	Che cosa fa
`nnet(blob)`	la rete di una stringa "BMNN" (formato in `src/ai/net.h` ; errore se è rotta), fino a 8 strati da 256
`net:run(inputs, [outputs])`	le uscite per una tabella di ingressi; `outputs` : una tabella da riempire invece di una nuova
`k, v = net:pick(inputs, [mask])`	l'indice (da 1) dell'uscita maggiore e il suo valore; `mask` : una tabella di booleani, `false` = quell'uscita non viene scelta
`n_in, n_out = net:size()`	quanti ingressi e quante uscite

Il pannello dell'assistente

`require "assist"` dà il pannello che gli strumenti aprono con un tasto (F6 nell'Assistant). Risponde mentre scrivi; Invio (A) passa il codice o lo sprite allo strumento, Esc (B) chiude, Tab (X) cambia modalità.

```
local assist = require "assist"

function _update()
  if assist.update() then return end          -- aperto: i tasti sono suoi
  local k = keyp()
  if k == "f6" then
    assist.open{ mode = "code", ctx = word_under_cursor,
                 on_insert = function(code) insert_lines(code) end }
  end
end

function _draw()
  draw_tool()
  assist.draw()                               -- sopra, se aperto
end
```

Funzione	Descrizione
`assist.open{...}`	apre il pannello. `mode` : `"code"` , `"sprite"` , `"mesh"` , `"error"` o `"any"` ; `query` , `ctx` , `error` , `size` e `palette` (sprite), `on_insert(code)` , `on_sprite(sprite)` , `on_mesh(m)` , `on_close()` , `x, y, w, h` (predefinito: quasi tutto lo schermo)
`assist.update()`	a ogni frame; vero finché il pannello è aperto e usa i tasti
`assist.draw()`	a ogni frame, sopra
`assist.is_open()` / `assist.close()`	chiede / chiude

Funzione	Descrizione
`assist.act(request, lines, n)`	che cosa chiede una riga `#entry: request #` alla riga `n` di `lines` : operatore ternario, commento, log, togliere i log, controllo di nil, rendere locale, indentazione, rinominare, ottimizzare, spiegare, un esempio da inserire o uno sprite scritto come codice. Restituisce `{lines, ok, message, cursor, explain}` ; non cambia nulla se non è abbastanza sicura

Completamento delle parole e scrittura con il pad

Funzione	Descrizione
`predict.complete(text_before_the_cursor, {lang = "lua"})`	`nil` oppure `{prefix, word, rest, ending, list}` : la parola più probabile che inizia come quella scritta. `lang` : `"it"` , `"en"` , `"lua"` , `"ask"` , un misto con pesi `( {it = 1, ask = 2} )` o `"none"` ; `words` i nomi del codice
`predict.count_words(lines)`	i nomi del codice, per `words`
`predict.preload({"lua", "it"})`	legge i dizionari un pezzo per frame, invece che alla prima parola
`pt.update(host)`	a ogni frame; legge `pad()` e cambia il testo tramite l'host `( before , insert , erase , newline , move , lang )`
`pt.text_host(lang)`	crea un host su una stringa
`pt.draw(x, y) / pt.size()`	disegna la sovrapposizione del controller / la sua dimensione
`pt.coach(host, text)`	il prossimo pulsante per scrivere un testo

`predict` è descritto in `docs/PREDICT.md` e `padtype` in `docs/PADTYPE.md` .

File di risorse e condivisione degli asset

Le risorse di un gioco stanno dentro le sezioni del suo `.bm`. Un file di risorse porta una risorsa fuori da un `.bm`, così puoi spostarla in un altro progetto.

I tipi di file

Estensione	Risorsa	Sezioni
`.bmm`	modelli 3D	MESH, ANIM (se animato), SHEET8 o SHEET (solo le celle usate dalle texture)
`.bmi`	immagini e sprite	SHEET8 o SHEET, SPRITES, BOXES, FLAGS
`.bms`	suoni	AUDIO
`.bmt`	mappa di tile	MAP, LAYERS, FLAGS, SHEET8 o SHEET (solo i tile usati dalla mappa)
`.bmc`	palette	SHEET8 di N x 1 pixel (N fino a 256)
`.bmk`	kit: più tipi insieme, un solo foglio condiviso	qualsiasi combinazione delle sezioni sopra

Che cosa fa

Un file di risorse usa lo stesso contenitore di un `.bm`, con il magic `BMRES` e tre byte a zero al posto di `BMCART`, e mai una sezione LUA. Il tipo all'offset 12 è 1 per `.bmm`, 2 `.bmi`, 3 `.bms`, 4 `.bmt`, 5 `.bmc`, 6 `.bmk`. Il tipo conta più dell'estensione: un file rinominato viene riconosciuto lo stesso. Ogni file può portare una sezione INFO.

Da sapere

- I nomi sono 8.3: l'estensione è `bm` più una lettera (mai `.bmp`). Il nome vero è dentro il file.
- Il menu non elenca i file di risorse tra i giochi, e i giochi non li leggono mentre girano. Una risorsa entra in un `.bm` per copia; la cartuccia resta un solo file che funziona da solo.
- INFO è testo, righe `chiave: valore`: `name`, `desc`, `author`, `license` (un id SPDX come `CC0-1.0`), `version`, `tags` (separati da virgole), `origin` (lo SHA-256 della risorsa da cui è stata copiata), `date` (AAAA-MM-GG). Un blocco `[tipo nome]` dà le chiavi di una parte; i tipi sono `model`, `sprite`, `sound`, `sfx`, `song`, `map`, `palette`. Al massimo 16 KiB; le chiavi sconosciute vengono conservate.
- Una zona di sprite si sposta nel foglio intera. In un gioco, `zspr(name, x, y)` la disegna e `zboxes(name, [frame, type])` legge i suoi box.
- I limiti sono quelli del `.bm`: foglio fino a 4096x4096, 256 modelli, 64 ossa.

Attenzione

- La scheda Lib, che elenca le risorse, per ora è nascosta. `lib_tab=1` in `bm/config.txt` la riporta.

- Le app che aprono i file di risorse non sono ancora pronte: A nella scheda Lib apre solo risorse che stanno dentro un `.bm`.
- `bmres.py add` con una cartuccia `.bme` e senza `-o` scrive il `.bme` sul posto. Con un `.bm` o un `.b16` e senza `-o` si rifiuta di cambiare il gioco ("is a game, read only") e scrive una copia con lo stesso nome e l'estensione `.bme`. Un'uscita `.bme` è segnata come progetto.
- [DA COMPLETARE: `docs/RISORSE.md` dice che `mkbm.py --res FILE` integra le risorse in fase di compilazione, ma in `scripts/mkbm.py` l'opzione `--res` sceglie la risoluzione dello schermo. Dire quale strumento fa l'integrazione in compilazione.]

Che cosa fa l'integrazione

Quando aggiungi un file di risorse a un `.bm`, `bmres.py` segue queste regole. Non toglie mai nulla dalla destinazione.

- Texture, sprite e tile prendono celle 8x8 libere del foglio di destinazione. Una cella è libera se è trasparente e nulla la usa. La cella 0 non si usa mai.
- Se non c'è posto, il foglio cresce in altezza, fino a 4096. La larghezza resta, così i numeri di sprite e mappa non cambiano. Un'isola più larga del foglio non ci sta, e un messaggio lo dice.
- Poi vengono corretti gli angoli delle texture, le zone SPRITES e i numeri della mappa.
- Le palette vengono unite. Oltre 256 colori il foglio diventa SHEET da solo.
- Se un nome esiste già si aggiunge un numero (16 caratteri per modelli e zone, 8 per i suoni).
- Uno scheletro segue il suo modello.
- I suoni vanno negli slot liberi.
- L'INFO della risorsa va nel blocco della voce, con `origin`.

Provalo, passo per passo: sposta un modello da un gioco a un altro

Ti servono Python 3 e il repository. I comandi girano sul PC, dalla cartella del repository.

1. Guarda che cosa contiene il gioco di partenza.
2. Estrai il modello in un `.bmm`. Usa il nome mostrato da `list`. Ottieni `VILLAGER.bmm`, con il suo scheletro e solo le celle di texture che usa.
3. Scrivi chi l'ha fatto e con quale licenza. Il file lo conserva a ogni copia.
4. Aggiungilo all'altro gioco. Usa `-o` per scrivere un file nuovo e tenere l'originale.
5. Esegui `list` sul risultato. Il modello c'è.

DA COPIARE

```
python3 scripts/bmres.py list GAME.bm
python3 scripts/bmres.py extract GAME.bm -o VILLAGER.bmm --model villager
python3 scripts/bmres.py info VILLAGER.bmm --name "Villager" --author me --license CC0-1.0 --tags charac
python3 scripts/bmres.py add OTHER.bm VILLAGER.bmm -o OUT.bm
python3 scripts/bmres.py list OUT.bm
```

PROVA

Estrai lo stesso modello due volte e aggiungi entrambi i file a un gioco. Al secondo viene aggiunto un numero al nome.

ATTENZIONE

Se `add` dice che un'isola non ci sta, il foglio di destinazione è più stretto dei pezzi. Non allarga il foglio.

Gli altri comandi

Il tipo di un nuovo file di risorse viene dall'estensione del suo nome. Le opzioni `--name`, `--author`, `--license`, `--version`, `--tags` e `--desc` funzionano anche con `extract` e `convert`.

DA COPIARE

```
python3 scripts/bmres.py extract GAME.bm -o OUT.bmi --sprite NAME
python3 scripts/bmres.py extract GAME.bm -o OUT.bmi --rect X,Y,W,H --name N --frames N --fps N
python3 scripts/bmres.py extract GAME.bm -o OUT.bms --sfx NAME --song NAME --sound NAME
python3 scripts/bmres.py extract GAME.bm -o OUT.bmt
python3 scripts/bmres.py extract GAME.bm -o OUT.bmc
python3 scripts/bmres.py extract GAME.bm -o OUT.bmk
python3 scripts/bmres.py convert IN OUT
python3 scripts/bmres.py convert IN.csv OUT.bmt --tiles TILES.png
```

`extract` senza `--sprite` e senza `--rect` prende tutto il foglio; senza le opzioni dei suoni prende tutto il banco dei suoni. `convert` va tra queste coppie:

Da	A
`.png`	`.bmi`
`.bmi`	`.png`
`.glb`	`.bmm`
`.json`	`.bms`
`.bms`	`.json`
`.bm` (un pacchetto di suoni)	`.bms`
`.hex`, `.gpl`	`.bmc`
`.bmc`	`.hex`, `.gpl`
`.csv`	`.bmt` (`--tiles`: il foglio a cui si riferiscono i suoi numeri)
`.bmt`	`.csv` (e il `.png` dei tile)

La struttura del progetto

Un gioco o un'app sta in `carts/<id>/`. La struttura comune tiene i metadati in un solo file, `bm.info`. Una cartella senza `bm.info` si compila ancora alla vecchia maniera, con titolo, autore e risoluzione dal Makefile, quindi puoi spostare un gioco alla volta.

```
carts/<id>/
  bm.info        metadati (sezione INFO), l'unica fonte
  main.lua       punto d'ingresso (sempre questo nome)
  cover.png      copertina del menu (facoltativa)
  assets/        sheet.png, map.csv, map_<layer>.csv, flags.csv, sprites.txt, models.bm|glb
  res/           file di risorse condivisi .bmm .bmi .bms .bmt .bmc .bmk
  build/         gli script di compilazione del gioco (mk*.py, build.py)
```

Che cosa fa

`bm.info` ha la stessa sintassi della sezione INFO: righe `chiave: valore`, blocchi `[tipo nome]` per le parti, e righe `#` che sono commenti e non vengono copiate. `mkbm.py --info` lo copia nella cartuccia.

Chiave	Significato	Usata dalla compilazione
`name`	nome mostrato	predefinito di `--title`
`author`	autore	predefinito di `--author`
`resolution`	`640x360`, `480x270`, `320x180`, `256x256`, `360x360`, `720x720`	predefinito di `--res`
`desc`, `license`, `version`, `tags`, `date`	come nella sezione INFO	solo conservate

Da sapere

- Il Makefile cerca prima in `assets/`, poi in `carts/<id>/`, così i giochi vecchi funzionano ancora.
- Le chiavi sconosciute vengono conservate nella cartuccia. Il formato binario `.bm` non cambia, e un kernel più vecchio ignora INFO.
- Giochi già spostati: `pong` (`bm.info`), `yharnam` (`bm.info`, `sheet.png` e `map_*.csv` in `assets/`, gli script `mk*.py`, `mapview.py` e `art/` in `build/`) e `hunt` (`bm.info`, `sheet.png` e `map.csv` in `assets/`, `mkassets.py` in `build/`).

Attenzione

- `--info` va con `--lua`; non si può usare con `--from`.
- Spostare file in `assets/` e `build/` cambia i percorsi negli `mk*.py` del gioco e in ogni documento che li nomina.
- Il `.bm` di un gioco spostato guadagna una sezione INFO di poche centinaia di byte. Per il resto è identico.
- La sua riga `title_<id>` esce dal Makefile.

Provalo, passo per passo: sposta un gioco nella struttura comune

L'esempio sposta un gioco chiamato `ball`, con un foglio e una mappa.

1. Crea `carts/ball/bm.info` con nome, autore e risoluzione del gioco (vedi il blocco sotto).
2. Crea la cartella `carts/ball/assets/` e spostaci `sheet.png` e `map.csv`.
3. Se il gioco ha script suoi (`mk*.py`), crea `carts/ball/build/` e spostali lì. Aprili uno per uno e correggi i percorsi verso `main.lua` e `assets/`.
4. Togli dal Makefile la riga `title_ball` del gioco, perché ora il titolo viene da `bm.info`.
5. Compila il gioco.
6. Confrontalo con il `.bm` che avevi compilato prima dello spostamento. Deve essere identico tranne la sezione INFO.

DA COPIARE

```
name: Ball
author: me
resolution: 640x360
license: CC0-1.0
version: 1
tags: demo
```

DA COPIARE

```
python3 scripts/mkbm.py -o ball.bm --lua carts/ball/main.lua --sheet carts/ball/assets/sheet.png --map c
```

ATTENZIONE

Se la compilazione non trova un file dopo averlo spostato, un percorso in uno dei tuoi script è ancora quello vecchio. Correggi il percorso; non rimettere il file dov'era.

Le schermate per gli sviluppatori

Un gioco o un'app salva come PNG sulla scheda SD il fotogramma che sta disegnando. Il giocatore lo fa con un tasto; il tuo codice lo fa con `shot()`.

Come	Dove
Print Screen sulla tastiera	`bm/shots/` sulla scheda SD
Select tenuto premuto per 3 secondi su un pad (Share su un DS4)	`bm/shots/` sulla scheda SD
`shot([prefix])` nel tuo codice	`bm/shots/` sulla scheda SD

Che cosa fa

Un riquadro dice `Screenshot saved` e il nome del file per due secondi. L'immagine è lo schermo del gioco com'era disegnato: le sovrapposizioni del dev kit, l'icona della batteria, il puntatore e quel riquadro non ci sono. Le prendono i giochi e le app avviati dal menu di bm; le schermate del menu stesso ancora no.

Da sapere

- `shot([prefix])` chiede il fotogramma che si sta disegnando. Viene salvato alla fine di `_draw`, quindi chiamalo in un punto qualsiasi del fotogramma; chiamarlo due volte in un fotogramma dà un solo file.
- Restituisce il nome che avrà il file, oppure `nil` e un messaggio se i numeri sono finiti. Non chiede nulla al giocatore.
- Il nome è `<PREFISSO><NNNN>.PNG`, 8.3. Il prefisso è formato dalle prime lettere o cifre del titolo del gioco, fino a 4 (`Pong` da `PONG`, `Hunter's Night` da `HUNT`, `SHOT` se non ce ne sono) oppure dal `prefix` che hai dato.
- Il numero è uno più del più alto di quel prefisso già presente in `bm/shots/`, da 0001 a 9999. Il Pi Zero non ha orologio, quindi non c'è la data.
- Il formato è PNG, 8 bit RGB, della dimensione dello schermo del gioco (640x360, 360x360...). Non si perde nulla: i colori a 16 bit della console vengono allargati a 8 bit. Non c'è il JPG, perché rovina la pixel art.
- Dal PC, sul collegamento di rete, `python tools/bm_net.py IP --shot out.png` chiede alla console uno screenshot del gioco in corso (la console chiede la password della Console) e lo salva come `out.png`. `python tools/bm_net.py IP --get bm/shots/PONG0007.PNG out.png` scarica un file già presente in `bm/shots/`. Il gioco deve disegnare fotogrammi: se non ne disegna, la richiesta fallisce dopo 3 secondi. Si possono scaricare solo file di `bm/shots/`. Non è verificato su un RGB30.
- Dal PC, `python tools/bm_net.py IP --play game.bm` invia un `.bm` alla console e lo avvia subito. La console chiede la password della console. Il file non viene salvato sulla scheda SD. Il menu avvia il gioco quando ha i tasti; se è in corso un aggiornamento o una pubblicazione sul Market, mostra un messaggio e non avvia il gioco. `--send game.bm` salva invece il file sulla scheda SD. Non verificato su un RGB30.
- Uno schermo piatto occupa pochi KB; il rumore può occupare quanto l'immagine grezza (circa 0,7 MB a 640x360).

Attenzione

- [DA COMPLETARE: verificare Select tenuto 3 s sul PowKiddy RGB30, e `shot()` con un gioco 3D sulla GPU; il repository dice che nessuno dei due è stato provato su un Pi o un RGB30.]

Provalo, passo per passo

1. Aggiungi le righe qui sotto al `_draw` di un tuo gioco e impacchettalo.
2. Avvia il gioco sulla console e premi **A** (o il tasto che usa il tuo gioco).
3. Aspetta il riquadro che dice `Screenshot saved` .
4. Metti la scheda SD nel computer e apri la cartella `bm/shots/` . L'immagine è lì.

[DA COMPLETARE: shots/api-tools-shot-saved.png]

PROVA

Chiama `shot("menu")` e `shot()` nello stesso gioco. Il primo file inizia con `MENU` , il secondo con le prime lettere del titolo.

DA COPIARE

```
local path = shot()          -- "bm/shots/PONG0007.png"
local path = shot("menu")    -- "bm/shots/MENU0001.png"
```

Per un manuale

Le immagini per un manuale si prendono sulla console, si copiano da `bm/shots/` nel repository e si nominano con un nome che dice che cosa mostrano. Sul PC, `bmhost --shots` e `scripts/qemu-screenshot.sh` fanno immagini senza console, e `make test-shot` controlla i PNG.

Punti aperti

- **Il file cartuccia**: stato del profilo `.b16` . `docs/B16.md` dice che il profilo è deciso ma non ancora implementato, e il §3 descrive un contenitore `B16CART` diverso che il §0 sostituisce con il contenitore `.bm` ; `src/bm/project.h` e `scripts/mkbm.py` segnano ed esportano già i file `.b16` (bit 1 all'offset 18, controllo degli 8 MiB). Dire quale delle due è quella attuale.
- **Il file cartuccia**: immagine da fare: shots/api-tools-editable-copy.png
- **File di risorse e condivisione degli asset**: `docs/RISORSE.md` dice che `mkbm.py --res FILE` integra le risorse in fase di compilazione, ma in `scripts/mkbm.py` l'opzione `--res` sceglie la risoluzione dello schermo. Dire quale strumento fa l'integrazione in compilazione.
- **Le schermate per gli sviluppatori**: verificare Select tenuto 3 s sul PowKiddy RGB30, e `shot()` con un gioco 3D sulla GPU; il repository dice che nessuno dei due è stato provato su un Pi o un RGB30.
- **Le schermate per gli sviluppatori**: immagine da fare: shots/api-tools-shot-saved.png